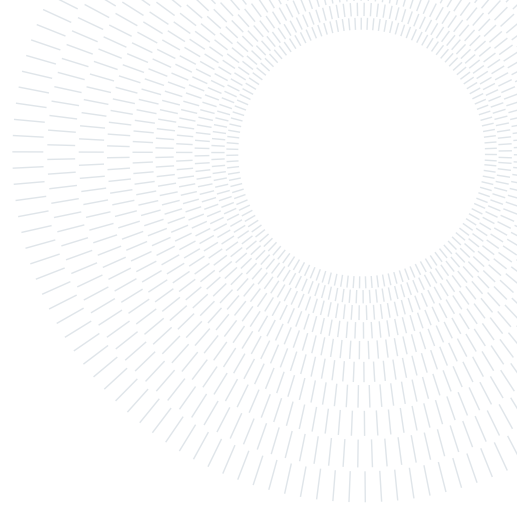




POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE



Deployment and Analysis of Kolmogorov-Arnold Networks on RISC-V with gem5

COURSE PROJECT IN
HPC ENGINEERING

Giovanni Vaccaro, Alessandro Tinaoui

Advisor:

Prof. Silvano Cristina

Co-advisors:

PhD Mattia Balla

Academic year:

2025-2026

Abstract: This work studies the deployment of Kolmogorov-Arnold Networks on a simulated RISC-V platform using gem5. The model is trained in Python, exported into a static representation, translated into C data structures, and finally compiled into a RISC-V executable. The goal is to connect the mathematical structure of the network with the concrete execution path of a single inference, from the exported parameters to the memory hierarchy observed in simulation.

1. Introduction

Kolmogorov-Arnold Networks (KANs) are a recent neural architecture built around learnable univariate functions. In a KAN layer, each connection transforms one scalar input through its own function, and each output node collects the contributions arriving from the previous layer. This makes the network naturally tied to function evaluation, spline representation, and memory layout. For this reason, implementing KAN inference on a simulated architecture is not only a matter of reproducing the numerical model, but also of deciding how its many small edge functions are represented in code and stored in memory.

The project presented in this report focuses on that implementation problem. A KAN model is trained in Python, exported into a representation that can be consumed by C code, cross-compiled for RISC-V, and executed inside gem5. The simulator is then used to observe the cost of inference at the architectural level, with particular attention to instruction execution, cache accesses, and the placement of model parameters in memory.

2. A Working View of Kolmogorov-Arnold Networks

2.1. Functions on edges

A KAN layer can be read as a collection of edge functions. For every pair of input node i and output node j , the layer contains a learnable univariate function $\phi_{i,j}$. This function receives only the scalar value x_i and

produces the contribution of that edge. The output node is then obtained by summing all contributions that arrive from the previous layer:

$$y_j = \sum_{i=1}^{n_{\text{in}}} \phi_{i,j}(x_i). \quad (1)$$

This equation is the most useful way to read a KAN for implementation purposes. Every edge is a small function evaluator, and every node is an accumulator.

The practical consequence is that the basic inference operation is the repeated evaluation of small scalar functions. For a CPU, and therefore for a simulator such as gem5, the cost of a KAN depends not only on the number of learned parameters, but also on how spline bases, coefficients, and temporary activations are stored and reused.

2.2. Edge parametrization and the NASA configuration

In a KAN layer, each edge does not store a single scalar weight. Instead, the edge from node i to node j represents a learnable univariate function $\phi_{i,j}$ applied to the scalar input x_i . For the implementation considered in this work, this edge function can be read as the combination of two branches:

$$\phi_{i,j}(x_i) = m_{i,j} \left[s_{i,j}^{\text{base}} b(x_i) + s_{i,j}^{\text{spline}} S_{i,j}(x_i) \right], \quad (2)$$

where $m_{i,j}$ is an optional edge mask, $s_{i,j}^{\text{base}}$ and $s_{i,j}^{\text{spline}}$ are learnable scale factors, $b(\cdot)$ is a fixed base activation, and $S_{i,j}$ is the learnable spline component.

Base branch. The term

$$s_{i,j}^{\text{base}} b(x_i) \quad (3)$$

is the base branch. It gives the edge a simple smooth response before the spline contribution is added. The activation function $b(\cdot)$ is fixed: its shape is not learned during training. What is learned is the scalar coefficient $s_{i,j}^{\text{base}}$, which decides how strongly this fixed response contributes to the edge.

In the NASA experiment, the base activation is SiLU:

$$b(x) = \frac{x}{1 + e^{-x}}. \quad (4)$$

SiLU is a smooth nonlinear function. For large positive inputs it behaves approximately like the identity, while for negative inputs it smoothly attenuates the value instead of applying a hard cutoff. In this context, the activation function is not applied after a whole layer. It is part of the edge function itself: it provides a stable low-complexity contribution, and the spline branch can then model more local corrections.

Spline branch. The term

$$s_{i,j}^{\text{spline}} S_{i,j}(x_i) \quad (5)$$

is the spline branch. This is the flexible part of the edge function. The shape of $S_{i,j}$ is learned through B-spline coefficients, so different edges can represent different scalar functions. The scale $s_{i,j}^{\text{spline}}$ controls the magnitude of this learned contribution.

Mask and scale factors. The two scale factors balance the two branches. If $s_{i,j}^{\text{base}}$ is large, the edge relies more on the fixed SiLU response. If $s_{i,j}^{\text{spline}}$ is large, the edge relies more on the learned spline shape. The mask $m_{i,j}$ acts as a gate on the whole edge: when it is zero, the edge contribution is removed; when it is one, the edge is active.

Initialization in the NASA model. For the NASA model used in this project, these quantities are initialized by the KAN library rather than manually overwritten by our training script. Since sparse initialization is not enabled, the mask is initialized to one for every edge and is not trainable. Therefore all edges start active and remain active in the exported NASA model. The base scale is initialized randomly for each edge in a range proportional to $1/\sqrt{n_{\text{in}}}$, and is trainable. The spline scale is initialized to $1/\sqrt{n_{\text{in}}}$ on each active edge and is also trainable. Consequently, after training, the NASA export contains both non-zero base scales and non-zero spline scales, so inference uses both the SiLU branch and the spline branch.

2.3. Spline representation

The spline term $S_{i,j}(x)$ is represented as a linear combination of B-spline basis functions:

$$S_{i,j}(x) = \sum_{r=0}^{M-1} c_{i,j,r} B_{r,p}(x). \quad (6)$$

The coefficients $c_{i,j,r}$ are the control points learned during training. The functions $B_{r,p}(x)$ are B-spline bases of degree p , and M is the number of control points associated with the edge. In this project, we use cubic splines, corresponding to degree $p = 3$.

The bases are defined over a knot vector

$$\mathbf{t} = [t_0, t_1, \dots, t_{K-1}], \quad (7)$$

which partitions the input domain into intervals. When the knot vector is uniform, the full vector does not need to be stored explicitly. If the first knot and the spacing are known, each knot can be reconstructed as

$$t_r = t_0 + r\Delta t. \quad (8)$$

Using this compact representation reduces static model data and simplifies the inference code.

2.4. Basis activation and locality

A degree-zero B-spline basis is an indicator function. It is active only on its own interval:

$$B_{r,0}(x) = \begin{cases} 1, & t_r \leq x < t_{r+1}, \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

Higher-degree bases are obtained through the Cox-de Boor recurrence:

$$B_{r,p}(x) = \frac{x - t_r}{t_{r+p} - t_r} B_{r,p-1}(x) + \frac{t_{r+p+1} - x}{t_{r+p+1} - t_{r+1}} B_{r+1,p-1}(x). \quad (10)$$

The key property is locality. At any input value x , a B-spline of degree p activates at most $p+1$ basis functions. For cubic splines this means that only four bases can be non-zero:

$$p = 3 \quad \Rightarrow \quad p + 1 = 4. \quad (11)$$

Therefore, even if an edge stores several control points, one inference step only needs the four coefficients whose support overlaps the current input value. The spline evaluation can be written as

$$S_{i,j}(x) = c_{i,j,q} b_0(x) + c_{i,j,q+1} b_1(x) + c_{i,j,q+2} b_2(x) + c_{i,j,q+3} b_3(x), \quad (12)$$

where q depends on the interval of the knot vector that contains x .

This observation is central from an implementation point of view. The spline branch of an edge can be reduced to a four-element dot product between local coefficients and local basis values:

$$\text{spline}(x) = \mathbf{c}_4^T \mathbf{b}_4(x). \quad (13)$$

As a result, optimization is not only a matter of reducing arithmetic operations. It is also a matter of placing the four coefficients close together in memory, avoiding unnecessary basis evaluations, and reusing the same local bases across all outgoing edges of the same input node.

2.5. Composition across layers

A full KAN is obtained by composing several layers. If $\mathbf{x}^{(\ell)}$ is the activation vector of layer ℓ , the next layer is computed as

$$x_j^{(\ell+1)} = \sum_{i=1}^{n_\ell} \phi_{i,j}^{(\ell)} \left(x_i^{(\ell)} \right). \quad (14)$$

The same formula applies independently of the input dimensionality. The first layer receives the input features, while later layers receive the activations produced by the previous layer. From the point of view of an inference program, the forward pass is a repeated pattern of edge evaluation and accumulation.

3. Pipeline of a Single Input Sample

3.1. From training to a RISC-V executable

Training is performed in Python, where the model is represented with PyTorch tensors and high-level KAN modules.

After training, the parameters are exported into an intermediate model file. This file contains the information needed to rebuild the forward pass outside Python: layer widths, spline degree, compact knot metadata, control points, scale factors, masks, and dataset-specific metadata such as input dimension or target scaling. The inference program does not read this file at runtime. Instead, a generation script translates the exported parameters into C headers containing static `const` arrays.

The offline path is therefore

$$\text{Python training} \rightarrow \text{model export} \rightarrow \text{C header} \rightarrow \text{cross-compilation} \rightarrow \text{RISC-V ELF binary.} \quad (15)$$

This choice keeps the simulated program small and deterministic. When `gem5` starts the executable, all model parameters are already part of the binary. There is no filesystem parsing, no JSON reader, and no dynamic model loading inside the measured inference path.

3.2. Where the model lives in memory

The generated arrays are declared as constant data. The compiler therefore places them in the read-only data section of the ELF binary, usually named `.rodata`. The executable is organized into the usual regions:

Region	Content	Role during inference
<code>.text</code>	compiled instructions	fetches through the instruction cache
<code>.rodata</code>	KAN coefficients and metadata	loaded through the data cache
<code>.data</code>	initialized mutable globals	used only when needed
<code>.bss</code>	zero-initialized globals	used only when needed
stack	temporary buffers and local variables	reused during the forward pass
heap	dynamic allocations, if present	avoided in the core inference path when possible

For this project, the most important distinction is between static model data and temporary runtime data. Spline coefficients, masks, knot metadata, and scale factors are stored in `.rodata`. Activation buffers such as `current[]` and `next[]` are runtime data and normally reside on the stack.

3.3. What `gem5` sees before inference

The simulations are run in syscall emulation mode. In this mode, `gem5` does not boot a complete operating system. It loads the ELF binary into the simulated process address space and starts executing it. At the beginning of the run, the program is present in simulated main memory, while the caches are initially empty:

```
simulated DDR3:
  .text
  .rodata
  .data
  .bss
  initial stack

L1I: empty
L1D: empty
L2:  empty
```

The caches are filled only as the simulated CPU fetches instructions and performs loads or stores. This is why the first accesses to model parameters and code blocks may generate misses, while later accesses can benefit from locality.

3.4. Forward pass of one sample

For a single input sample $\mathbf{x}$, the C implementation uses two activation buffers:

```
current[]
next[]
```

The buffer **current** stores the activations of the layer being processed. The buffer **next** accumulates the outputs of the next layer. At the end of the layer, the two roles are swapped or copied depending on the implementation.

The logical structure of the forward pass is:

```
current = input

for each layer:
    clear next
    for each input node:
        compute the local spline bases for current[input]
        compute the base activation for current[input]
        for each output node:
            read the edge parameters
            evaluate the edge contribution
            accumulate into next[output]
    current = next

return current[0]
```

This loop structure mirrors the mathematical definition of a KAN layer. Each input activation is reused across all outgoing edges. For this reason, computing the local spline bases once per input node and reusing them for all output nodes is more efficient than recomputing the same bases for every edge.

3.5. Path of one spline coefficient

Consider one spline coefficient stored as a **float**. It occupies four bytes in the generated model arrays. Before execution it is part of the ELF file; once `gem5` loads the program, it resides in the simulated main memory inside the `.rodata` region.

When the simulated CPU needs this coefficient, the data path is

```
simulated DDR3
-> L2 cache
-> L1D cache
-> CPU/FPU register
```

If the caches do not already contain the cache line, the load misses in L1D, then in L2, and finally reaches simulated memory. The memory system does not bring back a single float. It brings back a whole cache line. With a 64-byte line, one miss can load up to sixteen contiguous **float** values.

This detail explains why the memory layout of the model matters. If coefficients that are used together are contiguous, one cache miss can provide several useful values. If the layout is scattered or padded, the program may touch more cache lines than necessary. For a KAN, where many small edge functions are evaluated repeatedly, this effect can become as important as the arithmetic count.

3.6. Path of instructions and temporary data

The compiled inference functions are stored in `.text`. Their path through the memory hierarchy is separate from the data path:

```

simulated DDR3
  -> L2 cache
  -> L1I cache
  -> fetch/decode/execute

```

During inference there are therefore two main streams. Instructions flow from `.text` into the instruction cache. Model parameters and temporary buffers flow through the data cache.

The activation buffers and local basis values are small and repeatedly reused. After the first accesses, they are likely to remain close to the processor in L1D. The larger and more structured part of the memory traffic comes from the model arrays: coefficients, scale factors, masks, and metadata. This separation is useful when interpreting gem5 statistics, because instruction misses and data misses describe different aspects of the same inference.

3.7. Complete sample path

Putting the pieces together, the path of a model parameter is

$$\text{export} \rightarrow \text{C header} \rightarrow \text{RISC-V ELF} \rightarrow \text{simulated DDR3} \rightarrow \text{L2} \rightarrow \text{L1D} \rightarrow \text{register}. \quad (16)$$

The path of an input sample is

$$\mathbf{x} \rightarrow \text{current} \rightarrow \text{edge evaluation} \rightarrow \text{next} \rightarrow \dots \rightarrow \hat{y}. \quad (17)$$

The two paths meet inside the edge evaluator. The input activation selects the active spline interval, the local basis values are computed, the relevant coefficients are read, and the resulting edge contribution is accumulated into the next layer.

This view is useful because it connects the abstract KAN equations with the events measured by gem5. An optimization that removes unnecessary basis evaluations reduces arithmetic work. An optimization that stores uniform knots compactly reduces static data. An optimization that places edge parameters contiguously can reduce cache traffic. All three choices are part of the same deployment problem.

4. Implemented Work

This section summarizes the implementation work carried out in the project. The common goal of these changes is to move from a trained Python KAN to several executable inference variants that can be evaluated inside gem5.

4.1. From Training Artifacts to C Headers

The first implemented step was the export path from training to inference. The KAN is trained in Python, but gem5 executes a RISC-V binary, so the learned model must be converted into a representation that C code can include directly. For this reason, the training code writes the learned parameters to a JSON export. The inference scripts then read this JSON file and generate static C headers containing the network dimensions, spline coefficients, knot information, scale factors, masks, and dataset samples.

4.2. Isolating the Inference Kernel in gem5

The inference program was also modified to isolate the cost of the forward pass. In a naive benchmark, gem5 statistics would include initialization, warm-up, printing, target comparison, and metric computation. Those operations are useful for checking correctness, but they are not the KAN inference kernel.

The implemented flow separates these phases. The program performs setup first, optionally runs a small warm-up, resets gem5 statistics, executes only the inference loop, and then dumps the statistics before computing and printing the final metrics. This makes the reported cycles, instructions, and cache events much closer to the cost of the actual model evaluation.

4.3. Making Spline Evaluation Local

The floating-point C implementation was specialized for the structure used by the trained models. The project uses cubic splines, so for any input value at most four B-spline basis functions are active. Instead of evaluating and combining all basis functions of an edge, the optimized path finds the active interval and computes only the four local basis values. This reduces arithmetic work and also clarifies the future hardware direction: a cubic spline contribution naturally becomes a dot product between four basis values and four coefficients.

The representation of the knot vectors was also improved. When knots are uniformly spaced, the full knot vector does not need to be stored repeatedly. It can be reconstructed from a base value and a spacing. This reduces static model data and makes the inference code more regular.

4.4. Measuring the Role of the SiLU Branch

The NASA model also allowed us to study the role of the two contributions inside each KAN edge. In this configuration, an edge is not made only of the spline function. It contains a spline branch and a base branch, where the base branch applies the SiLU activation and then scales its contribution through `scale_base`. For this reason, removing the SiLU branch is not the same as disabling an edge mask. The mask would remove the entire edge, including the spline contribution. To isolate the effect of SiLU, we kept the masks unchanged and set `scale_base` to zero on every edge, leaving the spline branch active.

This ablation showed that the SiLU branch is very important for the NASA model. With the original model, which includes both the spline and SiLU branches, the full test-set inference obtained an MAE of about 9.6 and an MSE of about 191. After disabling the SiLU branch, the MAE increased to about 104.5 and the MSE to about 11812. The execution became lighter, because the program no longer evaluated and accumulated the base activation contribution, but the prediction quality degraded dramatically. This result suggests that, at least for the NASA dataset, the SiLU branch is not a small auxiliary term: it carries a substantial part of the learned mapping together with the spline component.

4.5. Quantization: From Real Values to Integer Codes

Quantization represents real-valued parameters and activations using integer codes plus a scale factor. The integer stores a discrete code, while the scale tells how much real value one integer step represents. Together, the pair “integer code + scale” approximates the original real value. For this reason, the process is not a simple cast to `int8` or `int16`: each value is first divided by a scale, rounded to an integer, and limited to the range allowed by the chosen precision.

For a real value r , the symmetric quantization rule used in this project is

$$q = \text{clip} \left(\text{round} \left(\frac{r}{s} \right), -q_{\max}, q_{\max} \right). \quad (18)$$

Here, q is the integer code, s is the scale, and $q_{\max}$ is the largest positive integer allowed by the chosen precision. For signed INT8, $q_{\max} = 127$. For signed INT16, $q_{\max} = 32767$. The operation first divides the real value by the scale, then rounds it to the nearest integer, and finally clamps it so that it fits in the available integer range.

As a simple example, assume that signed INT8 is used, so $q_{\max} = 127$, and choose a scale $s = 0.01$. A real value $r = 0.37$ is represented as

$$q = \text{round} \left(\frac{0.37}{0.01} \right) = 37. \quad (19)$$

The integer stored in memory is therefore 37, not the floating-point value 0.37. When the value is needed again, it is reconstructed as

$$\hat{r} = 0.01 \cdot 37 = 0.37. \quad (20)$$

If instead the value were $r = 1.50$, the same scale would give 150 before clamping. Since 150 is outside the signed INT8 range, it would be clipped to 127 and reconstructed as 1.27. This shows the main trade-off: a small scale gives accurate steps, but values outside the representable range saturate.

The approximate real value can be recovered as

$$\hat{r} = sq. \quad (21)$$

This recovered value $\hat{r}$ is not always exactly equal to the original r . The difference between them is the quantization error. A small scale gives fine precision, but it may saturate large values because the integer range is limited. A large scale covers larger values, but each integer step is wider and the approximation becomes coarser. Choosing the scale is therefore the central problem of quantization.

In our implementation, quantization is applied after training. The NASA KAN is first trained in floating point and saved as a normal checkpoint. Then, a post-training quantization script reloads the checkpoint and converts the learned parameters into integer representations. This conversion is applied to the quantities that are fixed after training, such as the spline coefficients, `scale_sp`, and `scale_base`.

Activations require a different treatment because they are not model parameters. A coefficient is known after training and can be quantized directly. An activation, instead, is created only while an input sample is being processed by the network. For example, the output of an internal KAN layer changes from one sample to another, so it is not available as a fixed array inside the checkpoint.

For this reason, the quantization script performs a calibration pass before the final inference export. It runs representative training samples through the model and records the range of the intermediate activations. In this context, $a_{\max}$ denotes the largest absolute activation value observed for a given layer during calibration. It is computed as the maximum magnitude, not as the largest signed value:

$$a_{\max} = \max_{\text{calibration samples}} |a|. \quad (22)$$

For example, if the observed activations of a layer range approximately from -2.10 to 1.80 , then $a_{\max} = 2.10$. Once this value has been estimated, the activation scale is chosen as

$$s_{\text{act}} = \frac{a_{\max}}{q_{\max}}, \quad (23)$$

where $q_{\max}$ depends on the selected precision. As a concrete example, with INT8 we have $q_{\max} = 127$. If calibration observes $a_{\max} = 2.54$, then $s_{\text{act}} = 2.54/127 = 0.02$. During inference, an activation equal to 1.00 is represented by the integer code 50 , because $1.00/0.02 = 50$. The real value can later be approximated again as $50 \cdot 0.02 = 1.00$. Therefore, parameters and activations follow the same quantization idea, but their scales come from different sources: parameter scales come from the trained tensors, while activation scales come from calibration data.

The complete flow can be summarized as follows.

Stage	Artifact	Role in inference
Training	Floating-point checkpoint	The NASA KAN is trained in FP32. This is the reference model.
FP32 JSON export	Floating-point JSON model	Stores floating-point coefficients, scales, masks, and normalization data.
FP32 header	<code>kan_model.h</code>	Generated from the FP32 export and included by the normal C inference path.
Quantized JSON export	QuantKAN JSON model	Stores integer parameter codes, parameter scales, and activation calibration scales.
Quantized header	<code>kan_model_quant.h</code>	Used by the quantized-dequantized C path: data are stored as integers but reconstructed and accumulated in floating point.
True-integer header	<code>kan_model_true_int.h</code>	Used by the true-integer C path: coefficients, activation codes, lookup tables, and requantization constants are arranged for integer arithmetic.
Test-data header	<code>nasa_test_data.h</code>	Contains the selected NASA test inputs and reference targets used by the RISC-V executable.